



THE PLAYBOOK

The AI Coding Agent Playbook.

A beginner-friendly guide to working with AI coding agents — from your first instruction to a reliable development workflow.

PROMPT ≠ WORKFLOW

EIGHT STAGES

ONE RUNNING EXAMPLE

GLOSSARY + REFERENCE CARD

Contents.

Thirty-two short chapters, one running example, and a workflow you can start using on your next task.

THE PROBLEM

02 WHY DOES AI STILL BREAK THINGS? Ch. 1

03 WHY THIS BOOK EXISTS Ch. 2

FOUNDATIONS

04 WHAT IS AN AI CODING AGENT? Ch. 3

05 CHATBOT VS CODING AGENT Ch. 4

06 THE BIG SHIFT Ch. 5

THE FRAMEWORK

07 THE AI CODING AGENT WORKFLOW Ch. 6

DEFINE

08 DEFINE THE JOB Ch. 7

09 DEFINE WHAT "DONE" MEANS Ch. 8

10 SCOPE AND CONSTRAINTS Ch. 9

CONTEXT

11 CONTEXT: AI NEEDS A MAP Ch. 10

12 PROJECT MEMORY Ch. 11

13 WHAT SHOULD YOU DOCUMENT? Ch. 12

THE AI CODING AGENT PLAYBOOK

CONTENTS

PART ONE

The workflow.

The problem with prompts alone, the foundations of working with agents, and the eight-stage system this book teaches.

WHY DOES AI STILL BREAK THINGS?



IN THREE LINES

- AI writes code fast, but often breaks things.
- It's usually not your prompt that's wrong.
- It's everything MISSING around the prompt.

Why can an AI coding agent write four hundred lines of code in ninety seconds — and still leave your project in worse shape than before you asked?

If you have spent any time with a modern AI coding tool, you have felt this moment. You typed a short request. The agent worked confidently. Files appeared. Code scrolled past. Something got installed. A message told you the task was complete.

Then you opened the app, and nothing worked.

Or worse: something that used to work now doesn't, and you can't tell what changed.

This is the strange paradox of the current moment. The tools are extraordinary. The results are unreliable. And most people quietly assume the problem is them — that they picked the wrong prompt, the wrong model, or the wrong tool.

WHY THIS BOOK EXISTS

IN THREE LINES



- Better prompts don't fix bad results.
- A better SYSTEM around the prompt does.
- This book teaches that system, step by step.

YOU TYPE

"Add rejected status"

AGENT



400 lines

BROKEN

- ✗ New page blank
- ✗ List broke
- ✗ Can't undo

WHAT WAS MISSING

Task · Scope · Boundaries · Definition of done · Verification · Review

The prompt wasn't wrong. Everything around the prompt was missing.

There are already thousands of articles about "the perfect prompt." There are prompt libraries, cheat sheets, and viral threads promising ten commands that will change your life.

They rarely change anything, because they treat the wrong layer of the problem.

WHAT IS AN AI CODING AGENT?

IN THREE LINES



- Chatbot = just talks.
- Assistant = suggests as you type.
- Agent = actually edits your project. Big difference.

CHATBOT

Answers questions



- ✓ Text in, text out
- ✗ No file access
- ✗ No commands
- ✗ No testing

*"Like a friend
on the phone."*

ASSISTANT

Suggests as you type



- ✓ Sees your file
- ✓ Autocompletes code
- ✗ Doesn't run code
- ✗ Doesn't edit files

*"Like autocomplete
with a brain."*

AGENT

Takes actions



- ✓ Reads whole project
- ✓ Edits many files
- ✓ Runs commands
- ✓ Runs tests
- ✓ Fixes its own errors
- △ Needs your rules

*"Like a junior dev
at your desk."*

The three kinds of AI helpers you can talk to. The right-most one is the topic of this book.

Before going further, it helps to be precise about what an AI coding agent actually is — because the term gets used loosely, and the differences matter.

CHATBOT VS CODING AGENT

IN THREE LINES



- Agents can do a LOT more than chatbots.
- That's why they cause bigger surprises.
- The more powerful the tool, the more your rules matter.

CHATBOT

Answers questions



- ✓ Text in, text out
- ✗ No file access
- ✗ No commands
- ✗ No testing

*"Like a friend
on the phone."*

ASSISTANT

Suggests as you type



- ✓ Sees your file
- ✓ Autocompletes code
- ✗ Doesn't run code
- ✗ Doesn't edit files

*"Like autocomplete
with a brain."*

AGENT

Takes actions



- ✓ Reads whole project
- ✓ Edits many files
- ✓ Runs commands
- ✓ Runs tests
- ✓ Fixes its own errors
- △ Needs your rules

*"Like a junior dev
at your desk."*

The three kinds of AI helpers you can talk to. The right-most one is the topic of this book.

The move from chatbot to agent is not a small upgrade. It is a change in kind. Comparing them side by side makes the shift, and its consequences, easy to see.

THE BIG SHIFT

IN THREE LINES



- Stop thinking: 'tell AI what code to write.'
- Start thinking: 'give AI a clear job with clear limits.'
- Same as hiring a plumber — you point at the leak.

A PROMPT

one instruction

"Fix the bug"

Life-span: minutes

Blast radius: one reply

Repeatable: no

Compounds: no

A WORKFLOW

a system around the instruction

Define · Context · Investigate

Boundaries · Execute · Verify

Review · Learn

Life-span: the whole project

Blast radius: every change (in a good way)

Repeatable: yes

Compounds: yes

Prompt is a sentence. Workflow is the working environment around that sentence.

Somewhere between your first week and your third with an AI coding agent, a quiet shift has to happen. Most people never make it, and that is the difference between the users who eventually ship real projects and the ones who quietly go back to copy-pasting snippets from a chatbot.

The shift is this.

THE AI CODING AGENT WORKFLOW

IN THREE LINES



- There are 8 steps to a healthy AI task.
- This page shows all of them at once.
- Each one gets its own chapter next.



DEFINE THE JOB

IN THREE LINES



- 'Improve the dashboard' is a bad instruction.
- Answer 4 questions first: what, where, result, rules.
- Then the agent has almost no room to guess wrong.

Almost every disappointing session with an AI coding agent starts the same way: the instruction was vague, so the agent filled in the blanks. It had no choice. Ambiguity does not stop an agent — it just gives it room to invent.

Look at this one, which sounds fine until you read it twice:

"Improve the dashboard."

Improve how? For whom? Which dashboard, if there is more than one? Faster, prettier, easier to read, more mobile-friendly, more accurate? Improve using what — a new library, a redesign, a rewrite? And what happens to everything that is already working on that page?

The agent doesn't know. So it decides. And whatever it decides is unlikely to match what was in your head.

A well-defined task answers four questions before any code is written.

1. Objective — what are you actually trying to achieve? Not the code. The outcome. *"So the student can tell at a glance which applications are*

DEFINE WHAT "DONE" MEANS



IN THREE LINES

- 'Done' isn't when the agent stops typing.
- It's when a real person could use it.
- Write a short checkable list — before you start.

Defining the job tells the agent what to do. Defining "done" tells the agent — and yourself — when to stop.

This sounds like the same thing. It isn't.

An agent will happily tell you a task is complete the moment it finishes typing. That is not the same as the task actually being complete.

"Complete" means the change works, the surrounding features still work, and the result matches what you asked for on a real screen, with real data, in the browser or app where the user will see it.

Without a written definition of done, every task drifts into a soft, agreeable ending. The agent says "Done!" You nod. You move on. Two hours later something is broken and neither of you can remember what changed.

A good definition of done is boring, specific, and checkable. You should be able to run through it like a short checklist and tick each line honestly.

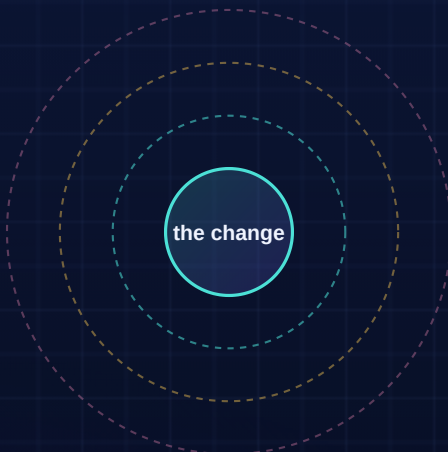
For the Applications-status task from the previous page, "done" might look like this.

SCOPE AND CONSTRAINTS

IN THREE LINES



- 'Blast radius' = how much can break if this goes wrong.
- Small scope = small blast radius.
- Name what to touch. Name what to leave alone.



Payments

Real money moves.

Authentication

Users get locked out.

Database schema

Every page that read it breaks.

Shared component

Other pages using it also change.

This one file

Only the thing you meant.

Before you push the button, ask: how far can this reach if it goes wrong?

There is a phrase professional engineers use when they talk about risky changes: **blast radius**. It means, if this change goes wrong, how much of the surrounding world gets damaged?

CONTEXT: AI NEEDS A MAP

IN THREE LINES



- The agent has never seen your project.
- Give it a map: folders, rules, commands.
- Save the map to a file. It reads it every time.

Even the most carefully defined task will fall apart if the agent has no idea what kind of project it just walked into.

Imagine hiring that same contractor from the earlier analogy — clear job, clear scope, clear definition of done — and then dropping them in front of a house they have never seen, with no floor plan, no idea where the water shutoff is, no notes about which walls are load-bearing, and no clue that the electrical panel is behind the laundry room door. They are still skilled. They are also, now, guessing.

That is what happens to an AI coding agent when you give it a good instruction inside a project it doesn't understand. It will still act confidently. It will just be guessing.

Context is the floor plan. It is everything the agent needs to know to make good decisions inside your specific project, as opposed to some average project it has seen versions of during training.

Useful context for a project like the Student Job Tracker includes things like:

PROJECT MEMORY



IN THREE LINES

- 'Project memory' is a file the agent reads first.
- Common names: AGENTS.md, CLAUDE.md.
- Write it once. Save yourself a hundred re-explanations.

Every time you open a new conversation with an AI coding agent, it starts from zero. It does not remember yesterday. It does not remember the last task. It does not remember the promise it made to never touch your authentication folder. Unless something reminds it, all of that context has to be reintroduced by you, by hand, every single time.

That is exhausting. It is also the reason many people quietly give up on agents after a few weeks. They mistake "I have to explain my project again" for "this tool doesn't work."

The fix is a very old idea in a new context: **write it down once, in a place the agent will always read.**

Most modern AI coding tools look for a specific file at the top of your project when a session starts. The exact name varies by tool, and the conventions will keep shifting. Common examples at the time of writing include:

- **AGENTS.md** — a growing shared convention across several agent tools.

- **CLAUDE.md** — used by Claude Code and some other Anthropic-facing

WHAT SHOULD YOU DOCUMENT?

IN THREE LINES



- In project memory: architecture, folders, commands, rules.
- NOT: generic advice, contradictions, outdated lines.
- Keep it under two minutes to read.

Project memory is only useful if it contains the right things. A bloated, contradictory, out-of-date file is worse than no file at all, because the agent will still read it and quietly follow instructions you no longer mean.

Aim for a short, honest document that answers the questions a competent new teammate would ask on their first day.

Include:

- **Architecture in one paragraph.** What is this project, in plain words? *"A single-page web app with a small local database. No separate backend. Deployed as static files."*
- **Important directories.** Where does each kind of thing live? Three or four lines is usually enough.
- **How to run and test it.** The actual commands. `npm run dev`, `npm test`, `npm run build`. If a command has a common gotcha (*"run `npm install` first if `node_modules/` is missing"*), name it.
- **Coding conventions that matter.** Not every stylistic preference — the

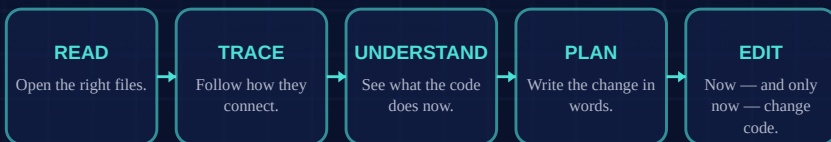
INVESTIGATE BEFORE EDITING

IN THREE LINES



- Bad idea: edit first, understand later.
- Good order: READ, TRACE, UNDERSTAND, PLAN, EDIT.
- Ask the agent to LOOK first. Every time.

LOOK BEFORE YOU LEAP



The five-step order every safe change follows. Edit is last.

There is a temptation, when working with an AI coding agent, to skip straight to the edit. You have a task. The agent is fast. Why wait?

Because the fastest way to break a project is to change code you do not yet understand — and that is true whether the person changing it is you, a junior developer, or an agent working at a hundred lines per minute.

Before any change, healthy work moves through five steps:

FIND THE RIGHT PLACE



IN THREE LINES

- Before writing code, find where it belongs.
- Ask the agent to search the whole project.
- This one habit prevents most 'oops' moments.

Every change you make lives somewhere. Part of working well with an agent is helping it — and yourself — figure out *where* before figuring out *what*.

First, a plain-English definition. A **codebase** is simply the collection of all the files that make up your project: the code, the styles, the configuration, the small helper files that glue everything together. When people say "search the codebase," they mean "look across all those files at once, not just the one you happen to have open."

Real projects are almost never a single file. Even a small app like the Student Job Tracker might have dozens: a page for the list of applications, a page for a single application, a form for adding one, a shared component for the status label, a file for talking to the database, a stylesheet, a configuration file, a test or two. A change to "how status works" could plausibly live in any of them. Often it lives in several.

Good agents, given a task, will do something like this before editing:

- **Search the codebase for related words.** "*Where does the word*

MAKE THE SMALLEST SAFE CHANGE

IN THREE LINES



- Big tasks fail in big ways.
- Small tasks are easy to test, review, and undo.
- Break every big goal into tiny working steps.

ONE BIG CHANGE

"Redesign the whole app"

- X Hard to read the diff
- X Hard to test
- X Hard to undo
- X When it breaks — where?
- X Never fully finished

1 giant leap

SMALL STEPS

- | | |
|-----------------------------|---|
| 1. Add status field to data | ✓ |
| 2. Show as plain text | ✓ |
| 3. Make it a colored label | ✓ |
| 4. Add filter buttons | ✓ |

4 tiny wins

Same feature. On the left: a nightmare. On the right: normal work.

There is a strong temptation, especially with an agent that can do so much so quickly, to hand over big tasks. *"Redesign the whole dashboard."* *"Refactor the data layer."* *"Rebuild the job tracker to look more like LinkedIn."*

EXECUTE IN SMALL STEPS

IN THREE LINES



- One step. Look. Next step. Look.
- Say: 'don't refactor anything I didn't ask for.'
- You're the pilot. The agent flies while you watch.

Once a task is defined, scoped, and understood, execution begins. This is the stage where most beginners hand the wheel entirely to the agent and hope. Don't. Execution is where your role shifts from planner to pilot — still not typing every line, but staying in the loop, one step at a time.

A healthy execution loop looks like this:

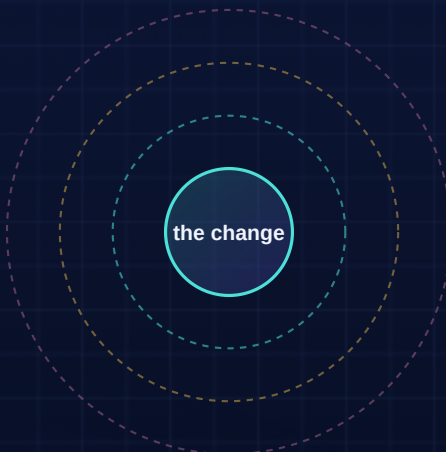
- **One logical change at a time.** Ask the agent to make a single, self-contained edit — not five. *"Add the `status` field to the data layer. Do not touch the UI yet. Stop when you're done."* A tight instruction produces a tight change, which produces a tight review.
- **Inspect the results before continuing.** Look at what changed. Open the app. Click the thing. Read the diff. If step one is wrong, do not build step two on top of it — you are now building on sand.
- **Avoid unnecessary refactoring.** Agents often tidy. They will rename variables, move helpers, and "improve" code that had nothing to do with your task. This is a hidden cost: a five-line change becomes a fifty-line change, half of which you didn't ask for and now have to review. **EXECUTE** Explicitly say: *"Change only what is needed for this task. Do not refactor*

UNDERSTAND THE BLAST RADIUS

IN THREE LINES



- Some tiny changes reach into big places.
- Auth, database, payments = high blast radius.
- Before every task: 'if this goes wrong, how bad?'



Payments

Real money moves.

Authentication

Users get locked out.

Database schema

Every page that read it breaks.

Shared component

Other pages using it also change.

This one file

Only the thing you meant.

Before you push the button, ask: how far can this reach if it goes wrong?

We introduced the phrase "blast radius" briefly on Page 10, when we first talked about scope. Now it deserves a page of its own, because it is the single most useful lens you can bring to any change an agent proposes.

SET EXPLICIT BOUNDARIES

IN THREE LINES



- Every task has TWO halves: do this, do NOT do this.
- Most people skip the second half. That's the problem.
- One magic sentence: 'if you need to touch X, stop and ask.'

Once you can see the blast radius, the next move is to make the boundaries of a task explicit — in writing, in the instruction itself. Not in your head. Not "the agent should know." In the actual sentences you send.

A good task instruction has two halves. The first says what the agent should do. The second says what the agent must not do. The second half is the one beginners almost always forget, and it is usually the more important of the two.

For the Student Job Tracker, adding a "status" field might look like this:

REAL-WORLD EXAMPLE

Change: - The Applications list component (`src/pages/applications/list.tsx`) and its styles. - The data function that reads applications from the database.

Do not change: - Authentication or anything under `src/auth/`. - The database schema (no new columns, no renamed columns). - Any file inside `src/payments/` or `src/config/`. - Unrelated components,

CODE THAT LOOKS RIGHT CAN STILL BE WRONG

IN THREE LINES



- Code that looks perfect can still be broken.
- Bugs, edge cases, side effects — none show at a glance.
- Never trust 'it looks right.' Actually test it.

Here is a hard truth about working with AI coding agents, and one that takes most people a few painful sessions to fully absorb:

Code that looks correct is not the same as code that works.

An agent can produce a change that compiles cleanly, uses good names, follows the project's conventions, and reads like something a thoughtful senior engineer would write — and still be quietly, completely wrong. This is not a failure of the agent. It is a property of code in general. Software has always been able to look right and behave wrong. Agents just produce more of it, faster, which means the gap between "looks right" and "is right" now matters more than ever.

The gap has a few common shapes. It helps to be able to name them.

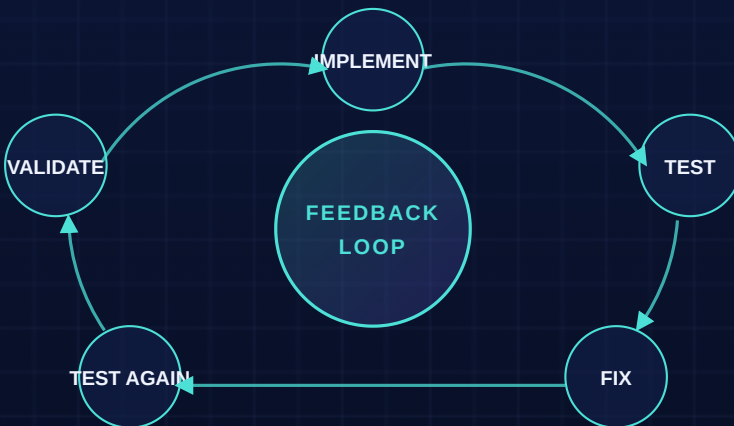
Bugs. The most familiar kind. The code does something, just not what you wanted. A filter that filters the wrong list. A save function that saves an empty record. A total that adds a number twice.

THE FEEDBACK LOOP

IN THREE LINES



- The loop: implement → test → fix → test → validate.
- A test is just asking the software 'does this work?'
- You can ask by hand or with code. Both count.



After every change: check. Then check the check. Until it really works.

There is a workflow shape that separates people who ship reliable software from people who ship confusing software. It looks like this:

WAYS TO CHECK THE WORK



IN THREE LINES

- Six ways to check the work — layer them.
- Manual clicks, tests, linting, types, builds, edge cases.
- Bigger change = more layers. Small change = fewer.

There is no single way to verify that a change is correct. Serious projects use several, layered on top of each other, so that different kinds of mistakes are caught by different kinds of checks. You do not need to master all of them at once. You do need to know what each one is, and roughly what it protects against.

Here they are, in plain English.

Automated tests. Small pieces of code that check other pieces of code. You (or the agent) write them once; then they run in seconds, every time, forever. A good test for the Job Tracker might be: *"when I save an application with status Active and then read it back, its status is still Active."* If someone later breaks the save function, that test fails immediately.

Linting. A tool that reads your code and complains about small stylistic or structural problems — unused variables, inconsistent formatting, obvious mistakes like using a variable before defining it. Think of it as a spell-checker for code. It catches nothing about whether the software *does the right thing*, but it catches a lot of noise that would otherwise slow down review.

TESTS AS THE REFEREE



IN THREE LINES

- Tests are the referee. They don't lie.
- Passing tests $\neq$ perfect. Only what YOU tested passes.
- Never soften a test to make it pass. Fix the code.

Of all the ways to check the work, automated tests deserve a page of their own — because they play a specific role that nothing else can play.

Tests are a **referee**. They are the piece of your project that gives an objective, repeatable answer to the question "does this still work?" — untouched by anyone's opinion or memory or hope.

That role matters more with AI coding agents than it ever did with humans alone, for a simple reason. Agents move fast and sound confident. When an agent tells you a change is complete, you have three options: trust the claim, spend time reviewing every line, or run a test that answers the question directly. Only the third scales.

Tests reduce guesswork. Without them, "does the save function still work?" is answered by *trying to remember* whether the last change might have affected it. With them, it is answered by running one command and reading one word: passed, or failed.

Tests increase confidence when they pass. If every test in the Job Tracker passes after the agent's change, you know — not hope — that all the

REVIEW THE DIFF

IN THREE LINES



- A 'diff' shows exactly what changed. Red out, green in.
- Bring 6 questions when you read one.
- Never ship a change you didn't read.



applications/list.tsx

```
10 function ApplicationList() {  
- 11 return <ul>{apps.map(...)}</ul>  
+ 11 const [f, setF] = useState('all')  
+ 12 const visible = apps.filter(byF(f))  
+ 13 return <><Filter on={f}/><List rows={visible}/></>  
14 }
```

RED = removed

GREEN = added

Read each line before you approve.

A diff shows what actually changed — red went out, green came in.

Before we can talk about reviewing a change, one piece of vocabulary is worth naming clearly: the **diff**.

A diff is simply a side-by-side view of what a change actually did. Not what the agent said it did. Not what you asked for. What the files, right now, look like compared to what they looked like a few minutes ago. **REVIEW**

THE HUMAN CHECKPOINT



IN THREE LINES

- AI is fast. YOU are accountable.
- Product choices, security, final approval = yours.
- You are the pilot. Not the passenger.

A useful way to think about your role, once you have built a real workflow around an agent, is this: **the agent is allowed to be fast; you are required to be accountable.**

Speed and accountability are different jobs. Modern agents can absorb the speed job entirely. Nothing and nobody can absorb the accountability job for you. It is worth being specific about what that means in practice — what actually stays on your side of the line, no matter how capable the tools become.

Product decisions. What should this software do? Who is it for? Which features matter now, which can wait, and which are quietly bad ideas dressed up as reasonable requests? An agent can propose. It cannot decide, because deciding requires knowing what you and your users actually want, and the agent does not.

Architecture decisions. How the pieces of the project fit together. Whether this feature belongs in a new file or an existing one. Whether a small bit of duplication now is preferable to a premature abstraction. **These** decisions echo for months. They should be made by someone who will still

TURN MISTAKES INTO RULES

IN THREE LINES



- Every surprise = a new rule.
- Write the rule in project memory.
- Same mistake won't happen twice.

MISTAKE

Agent changed a file outside the feature.

NEW RULE

"Don't touch files outside the feature. Ask if unsure."

NEXT TIME

Agent stays in scope. Mistake never returns.

Write it once. Every future session gets it for free.

Every surprise is free training data — if you write the rule down.

Every time you work with an AI coding agent, something small goes wrong. The agent modifies a file it should not have. It invents a function name that doesn't exist. It picks a library you don't use. It "cleans up" code you liked the way it was. It over-refactors. It renames things without being asked.

Beginners react to each of these moments in isolation. They fix it, they sigh, they move on, and next week the same thing happens again. That is the trap.

MAKE THE PROJECT BETTER AT WORKING WITH AI

IN THREE LINES



- A good project gets EASIER over time, not harder.
- Every session, leave better docs, tests, rules behind.
- Same AI + better project = way better results.

There is a quiet, cumulative property to a well-run project that uses AI coding agents, and it is one of the most important ideas in this book. Stated plainly:

A project should get easier to work with over time, not harder.

Most codebases go the other direction. They start clean and small. Features pile on. Conventions drift. Documentation goes out of date. Nobody remembers why a particular file exists. Every new change costs more than the last, and every new person — human or agent — takes longer to become useful. This is entropy, and it happens by default.

Working carefully with an AI coding agent gives you an unusual opportunity to fight that entropy on purpose. Every session becomes a small chance to leave the project slightly better organized, slightly better explained, and slightly easier for the next task. Over months, the compound effect is dramatic.

Concretely, a well-tended project accumulates:

THE COMPLETE AI CODING AGENT OPERATING SYSTEM

IN THREE LINES

i

- The whole system on one page.
- 8 stages. Each one prevents a specific failure.
- When a task fails, look here to spot which stage you skipped.



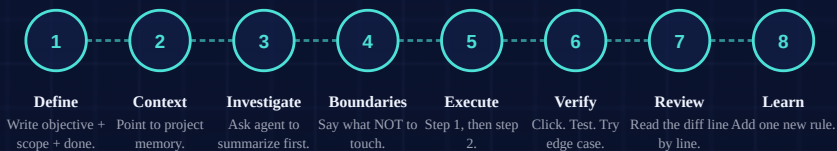
REAL-WORLD WALKTHROUGH

IN THREE LINES



- One real task. All 8 stages. Start to finish.
- Watch how a vague wish becomes shipped code.
- This is what 'controlled autonomy' looks like in practice.

ONE FEATURE · EIGHT STOPS



Each stage is small. Together they turn a wish into a shipped feature.

To make the operating system concrete, here is one realistic task in the Student Job Tracker, worked all the way through. The task is small, deliberately. A small task, done well, teaches the workflow more clearly than a large task done impressively.

The task, as it might first appear in your head: "Add a filter to the dashboard so I can see only the applications with a given status."

PART TWO

Practice & reference.

A reusable task template, the two working checklists, and a closing chapter on the only skill in this field that will still be worth having in ten years.

REUSABLE AI CODING TASK TEMPLATE

IN THREE LINES



- A fill-in-the-blanks template you can steal today.
- Fill it before every task, until it's automatic.
- You're building your own operating manual.

AI Coding Task · fill in the blanks

TASK	<i>One-line name.</i>
GOAL	<i>The outcome — why it matters.</i>
CONTEXT	<i>Which folders / files.</i>
INSPECT FIRST	<i>Ask the agent to look before it leaps.</i>
SCOPE	<i>What may change.</i>
BOUNDARIES	<i>What must NOT change.</i>
SUCCESS CRITERIA	<i>Checkable list of what 'done' looks like.</i>
VALIDATION	<i>How you'll prove it works.</i>
REVIEW	<i>What you'll read in the diff.</i>

THE TWO FINAL CHECKLISTS



IN THREE LINES

- Two checklists: before you ask, and before you accept.
- Plus a list of RED FLAGS that mean 'stop.'
- Use them the way a pilot uses a pre-flight checklist.

You have the operating system. You have the template. This page collects the two shortest possible checklists — one for before you ask the agent to work, one for before you accept its work.

Use them the way a pilot uses a pre-flight checklist. Not because the pilot has forgotten how to fly, but because forgetting one small thing at speed is how accidents happen.

Checklist 1 — BEFORE ASKING THE AGENT TO WORK

- [] I can state the **objective** of this task in one sentence.
- [] I have written **scope**: which files, pages, or features may change.
- [] I have written **boundaries**: what must not change, including any sensitive areas nearby.
- [] I have described the **expected result** in terms a user would recognize.

- [] I have named the **constraints** that must stay true (no new libraries,

THE FINAL PRINCIPLE

IN THREE LINES



- Tools will change. Your habits will not.
- The workflow is the only skill that compounds.
- Don't chase max autonomy. Aim for CONTROLLED autonomy.

Tools change. Models change. The interface you use today will look old within a year, and the interface you use next year will look old the year after that. Whatever specific product name is on the tab in your browser right now — Claude Code, Cursor, Copilot, Codex, Gemini, something newer — will be joined, replaced, or subsumed by another.

Almost nothing in this book depended on which one you use.

What this book asked you to build is portable, because it lives in *you*, and in the small habits you leave behind in your projects. A short project-memory file. A task template you actually fill in. A definition of done that is checkable. A boundary line that says *stop and ask*. A diff you read before moving on. A rule captured after every surprise.

These are humble artifacts. None of them will trend on social media. Together, they are the difference between someone who is technically using AI and someone who is actually shipping.

Glossary.

Plain-English definitions of every technical term used in this book.

Reference card.

Tear this out. Keep it next to your keyboard. Or just remember it.

Tear this out. Keep it next to your keyboard. Or just remember it.

THE ONE-LINE THESIS

A prompt is one instruction. A workflow is the system around it. Good results come from good systems, not clever sentences.

THE EIGHT-STAGE OPERATING SYSTEM

DEFINE



CONTEXT



INVESTIGATE



BOUNDARIES



EXECUTE



VERIFY



REVIEW



LEARN

THE AI CODING AGENCY PLAYBOOK

1. **DEFINE** — objective, scope, expected result, constraints, done. **REFERENCE**

2. **CONTEXT** — project memory: what this project is and how it works.